Trust-minimized Bridging Between Federated Byzantine Agreement Blockchains

CHRIS FROST, GIULIANO LOSA, and DAVID MAZIÈRES
Stellar Development Foundation, USA

Federated Byzantine Agreement (FBA) is an open-membership consensus model in which nodes determine quorum composition in a decentralized way. Specifically, each node chooses sets of nodes known as *quorum slices*, and a quorum must contain at least one slice from each of its members. In this paper, we show that the notion of quorum slice can be extended across instances of an FBA protocol, providing a means for nodes in one FBA system to agree on a recent state of another. This capability makes it possible to move assets between FBA blockchains without entrusting the custody of escrowed assets to any external third parties.

1 Introduction

Blockchain bridging allows assets to be used across multiple blockchains. Typically, an asset is locked up on one chain in exchange for releasing or minting a proxy asset on the destination chain. To move in the other direction, the original, locked asset is released in response to destroying or returning the proxy asset. The key challenge for bridging is allowing logic on one blockchain to verify that something has happened on the other blockchain so as to determine when to release or mint assets.

The simplest bridges employ trusted third parties or committees to release locked assets as appropriate. Unfortunately, such architectures can suffer the theft of locked funds even when the underlying blockchain consensus remains secure [2, 3], rendering the bridge insolvent. The best practice is therefore to employ *trust-minimized* bridging, in which one blockchain directly verifies the consensus output of another blockchain.

Proof-of-work (PoW) blockchains can achieve trust-minimized bridging if each blockchain verifies the cumulative work behind the other's block headers. This verification can be made practical through techniques such as Non-interactive Proofs of Proof of Work [7]. Unfortunately, PoW lacks asymptotic security against a computationally powerful adversary, which particularly undermines blockchains other than Bitcoin [4]. As such, PoW has fallen out of favor for new blockchains.

Proof-of-stake (PoS) blockchains can achieve trust-minimized bridging by ingesting enough of each other's state to track valid signers and verify new block headers. Cosmos IBC [6] demonstrates the approach. Unfortunately, PoS is complicated by long-range attacks, in which validators who have withdrawn their stake can sign alternate histories without risking slashing. Hence, while consensus protocols typically remain live under partial synchrony, IBC with PoS requires human intervention following a sufficiently long network outage.

A third approach to blockchain consensus is Federated Byzantine Agreement (FBA) [9], introduced by the Stellar [8] blockchain. Whereas PoW and PoS use cryptocurrency rewards to incentivize potentially anonymous validators to behave correctly, FBA leverages the real-world reputation of validator operators for security. Operators configure FBA validators to guarantee agreement with other validators based on the need to interoperate with other organizations. Security depends on the so-called *Internet hypothesis*—roughly, the assumption that any two validators one cares about will have some common transitive dependency that prevents them from diverging.

This paper presents a technique for trust-minimized bridging between FBA blockchains. Unlike prior settings, FBA bridging can enjoy both asymptotic security and liveness under partial synchrony. We assume an IBC-like protocol in which validators embed light clients for foreign blockchains. However, unlike PoS, each validator individually configures its light client to ensure agreement

Draft of October 27, 2026.

with specific sets of validators of the foreign blockchain. This "quorum in the eye of the beholder" model naturally extends the FBA trust model across networks.

2 Model

This section describes the blockchain model, the federated Byzantine agreement (FBA) model of consensus, and the model for FBA bridging.

2.1 Blockchain model

A blockchain is a replicated state machine. A set of replicas use a consensus protocol to agree on a log, then each deterministically apply the log to update their copy of the state, typically a ledger. Each log entry is known as a *block*, as it generally contains a set of transactions that atomically all commit together when the block is appended to the log. What distinguishes a blockchain from an ordinary replicated log is that consensus takes place on a *block header*, which contains a collision-resistant hash of both the current block and the previous header (except for the first block). Hence, agreement on a block header implies agreement on both the current block and all previous blocks.

Blockchains with smart contracts typically support output *events*. One can implement a bridge by processing events on a foreign blockchain: for every event indicating that an asset was locked or destroyed by a bridge contract on the foreign blockchain, the local bridge contract can release a corresponding local asset. We assume that events emitted on a foreign blockchain can be verified in a local smart contract by hashing given the remote block header and a small amount of auxiliary data such as a Merkle path. (If all bridge events from a given do not fit in calldata, one smart contract invocation can at least verify and process a range of events from the block.) The key functionality we need for trust-minimized bridging is for smart contracts on one blockchain to be able to verify block headers from a foreign blockchain.

2.2 FBA consensus model

In traditional Byzantine agreement systems, a quorum is a set of validators that is sufficient to make a decision. For example, a protocol for n validators may stipulate that unanimous votes from more than two thirds of the n validators trigger a decision. In this case, every set of validators containing more than two thirds of the validators is a quorum.

Unlike traditional Byzantine agreement, FBA systems determine quorum composition in a decentralized manner. Each validator v of an FBA consensus protocol chooses a set of *quorum slices*, where v only recognizes a set Q as a *quorum* when (a) Q contains at least one slice of v and (b) for every validator v' in Q, Q contains at least one slice of v'. One way to understand this definition is to interpret slices as specifying agreement requirements: a quorum is a set that satisfies the agreement requirements of all its members, and a validator recognizes a quorum when that quorum satisfies the validator's own requirements.

As an example, an FBA system with n nodes can mimic a centralized Byzantine agreement protocol if all validators accept any $\lfloor 2n/3 \rfloor + 1$ nodes as a quorum slice. Alternatively, a validator that wants to agree with 3 specific organizations, each of which runs 3 validators, can choose its quorum slices to include 2/3 of the validators of each organization. Larger quorum slices increase safety at the cost of decreased liveness; at the extreme, choosing a single giant quorum slice containing every validator in the world guarantees agreement with everyone while blocking progress in the face of a single unavailable validator.

Every message broadcast by a validator is digitally signed and specifies the sender's quorum slices (e.g., as a collision-resistant hash). Since validators may choose a large number of quorum slices, protocols should encode slices in a more restricted but compact way. For example, Stellar uses "quorum sets", which specify that any k-of-n items constitute a quorum slice, where each item

is either a validator public key or, recursively, another quorum set. Technically, since slices are specified in messages, the notion of *quorum* applies only to a set of messages—e.g., a quorum of votes for a particular outcome. However, since validators rarely change quorum slices, we often abuse the term to refer to message senders, as in "a quorum voted for the same outcome."

2.3 FBA bridging model

For each foreign blockchain bridged to the local blockchain, we assume that every local validator includes a *light client* of the foreign blockchain. Unlike Cosmos IBC light clients, which need to understand proof of stake, FBA bridge light clients need to know almost nothing about the structure of foreign blocks—just how to check their parent relationships and how to verify signatures by foreign validators on "externalized" headers. Every light client has a set of quorum slices for the foreign blockchain. That means each validator operator must choose one set of quorum slices for the local blockchain and one set of quorum slices for each foreign blockchain.

The light client maintains a sequence of *imported* block headers from the foreign blockchain—where imported roughly means that a local quorum believes the header is valid for the foreign blockchain. Whenever the local blockchain is assembling a block to propose to consensus, it asks the light clients to supply the most recent imported block header for each foreign blockchain. A local block that has been decided through consensus references block headers of foreign blockchains and cannot be externalized at a node until the node believes all those foreign references are valid.

To enable bridging, we extend the runtime API to allow smart contracts to ask two questions of light clients:

(1) Is c_i an ancestor of the current block's foreign block header for foreign blockchain C?
(2) Is c_i the parent of c_{i+1} on foreign blockchain C?

Both questions may seem trivial—after all, shouldn't a foreign block header include a collision-resistant hash of its parent? The problem is that blockchains are not perfect; occasionally they have suffered consensus failures, non-determinism bugs, or logic errors and been unable to recover without rewriting previously externalized history, an occurrence known as a *reorg*. For example, divergent database lock limits once famously led Bitcoin to discard 53 externalized blocks; Bitcoin reexecuted most of the discarded transactions in the reorganized history, but one double-spend had to be rejected [1].

Once a local smart contract has seen a block header c for a foreign blockchain C and processed the corresponding events, the effects cannot be locally rolled back even if c is discarded in a reorg of C. Recovering from a foreign reorg therefore requires a way to splice locally externalized but remotely discarded blocks, such as c, into the newly reorganized history of C. This can be done by hard-coding a special *splice* rule into light clients that overrides the normal hash check to consider one specific block header to have a parent that differs from the hash value in the header. Inserting such a rule is akin to a protocol upgrade of the local blockchain, which in FBA systems typically goes through local consensus.

Blockchain reorgs *should* never happen. Unfortunately, historically they have, so we need to anticipate that node operators will need to correct them. To be clear, splicing cannot recover lost funds in the event that a reorg reversed deposits into a foreign bridge contract after the corresponding local tokens were released. Blockchain communities need to repair the damage from reorgs as responsibly and fairly as possible—e.g., possibly finding a way to recapitalize a bridge that has lost funds. The point of splice rules is not that they magically mitigate reorg damage. Rather, the point is to limit the scope of whatever extreme recovery measures people take, so as to ensure that a foreign blockchain reorg does not affect local smart contracts that don't care about the foreign blockchain or its assets.

Bridges to foreign blockchains cannot affect the safety of the local blockchain. They should also minimally affect liveness. Unfortunately, there are specific situations in which you actually want a bridge to prevent the local blockchain from producing new blocks, namely when there's a fork or failure of the foreign blockchain and local validators genuinely disagree over the correct state of the foreign chain. That said, three ingredients are all required for a bridge to prevent new blocks: there must be a foreign validator failure or misconfiguration, there must be a local validator failure, and there must be divergence in the configuration of the local blockchain's light clients. Section 4 discusses some subtleties around the liveness of otherwise non-faulty nodes that have minority light client configurations. Importantly, however, liveness can be protected through norms ensuring identical or highly similar light client configurations at all local validators.

3 Protocol background

Most consensus protocols can be decomposed into a series of *votes*. For optimization reasons, the votes may not be explicit in the protocol messages—for instance, a single message may conceptually cast votes for multiple statements, or, because FBA voting requires two phases, a single message may implicitly piggyback the first phase of one vote onto the second phase of another vote. Nonetheless, to understand consensus protocols, it often helps to think of them in terms of an equivalent conceptual protocol in which all votes are explicit.

The purpose of a vote is to decide among mutually exclusive outcomes. Each non-faulty node *votes* for at most one candidate outcome as input, and may subsequently see a decision outcome as output. When a vote is successful and certain failure assumptions hold (e.g., an n-node Byzantine fault tolerant protocol typically assumes fewer than $n/3$ nodes have failed), all non-faulty nodes see the same decision outcome. Unfortunately, voting can also get permanently stuck if nodes vote for different outcomes, such that no single outcome is supported by a quorum. Worse, with node failure and unknown network delay bounds, it can be impossible to determine whether a vote is permanently stuck or will eventually succeed: a missing vote message could be delayed by the network and eventually complete a quorum, or might never have been sent if the sender failed.

Under the appropriate failure assumptions, voting enjoys the following properties:

- **Non-triviality.** If a non-faulty node decides an outcome, at least one node voted for that outcome.
- **Validity.** If a non-faulty node decides a value, the value is semantically valid at that node.
- **Consistency.** All non-faulty nodes that decide an outcome decide the same outcome.
- **Unanimity.** If all non-faulty nodes unanimously vote for the same outcome, the vote will reach a decision and not get stuck.
- **Totality.** If a vote produces a decision at one non-faulty node and the value is semantically valid at all non-faulty nodes, the vote will produce a decision at all non-faulty nodes.

Non-triviality and validity can easily be enforced at individual nodes. Consistency requires *quorum intersection*—the property that any two quorums share at least one non-faulty node. Unanimity holds under *quorum availability*—the property that there exists at least one entirely non-faulty quorum. In a centralized Byzantine agreement setting, totality can be achieved by broadcasting *quorum certificates*—vote messages digitally signed by a quorum. In the FBA model, different nodes accept different quorums, so there is no notion of a quorum certificate acceptable to all nodes. Achieving totality in the FBA setting requires a two-round federated voting protocol.

3.1 Federated voting

In the FBA model, well-behaved nodes that faithfully obey the protocol can still fail if their quorum slices are misconfigured and allow disjoint quorums. In practice, strong norms [5] prevent this

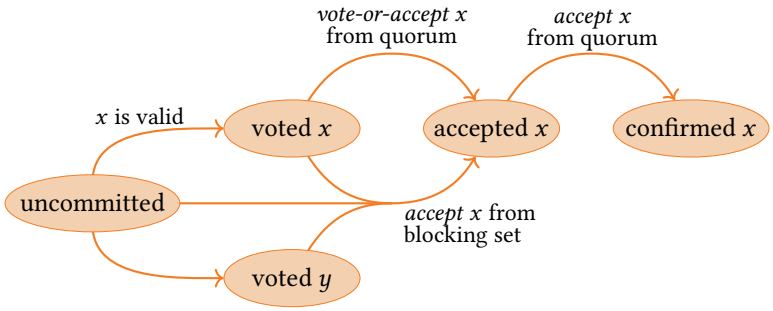


Fig. 1. Stages of federated voting

from happening to any validators that matter. Someone could deliberately spin up a set of new, misconfigured validators to create a fork, but no one who follows any of the existing validators would accept any blocks on the branch of the new validators. Given the possibility of disagreement over which well-behaved nodes have faulty configurations, properties such as validity and consistency become properties of *sets of nodes* in FBA, not properties of the whole system.

Federated voting guarantees validity and consistency among *intertwined* sets of nodes, and guarantees unanimity and totality among *intact* sets of nodes. We define these terms precisely in Section 5.2. Informally, nodes are intertwined when any two of their quorums share a well-behaved node, which is essentially a relativized notion of quorum intersection. An intact set is a quorum of well-behaved nodes that remains intertwined even if all non-intact nodes fail. The existence of an intact set is analogous to but stronger than quorum availability; it accounts for the fact that well-behaved nodes can still get stuck in incompatible states if they allow disjoint quorums.

Unfortunately, no single-round voting protocol achieves totality in an FBA setting, because nodes may not even know of the existence of certain other non-faulty nodes, much less what those nodes will accept as a quorum. Moreover, because nodes can adjust their quorum slices unilaterally at any time, even a set of votes that appears to be a valid quorum certificate for another node may no longer be. On the other hand, a node v always knows its own quorum slices, and so knows when enough nodes disagree with it that v can't be in an intact set that decides some value y. At a high level, then, federated voting uses a first quorum to establish that if v is intact, then it can vote for some value x without getting stuck. A second quorum then ensures that intertwined nodes agree on x even when they are not intact.

Consider a set of nodes S that includes at least one node from every one of node v's quorum slices. If every node in S is faulty, v cannot be a member of an intact quorum, since every one of v's quorums will contain at least one node from S and all nodes in S are faulty. We say that a set such as S is *v-blocking* if S intersects every quorum slice of v, meaning that v cannot be intact unless at least one member of S is non-faulty.

Figure 1 depicts the federated voting protocol at each node v. First, v *votes* for one among several mutually exclusive outcomes (abstractly x or y in the diagram) by broadcasting a digitally signed vote message including a hash of v's quorum slices. If v is part of a quorum that unanimously votes for some value x, then v *accepts* x and broadcasts a new message declaring it has accepted x, once again including a hash of its quorum slices. Whether or not v voted for x, if a v-blocking set has unanimously accepted x, v also accepts x and broadcasts this fact. The fact that a v-blocking set has accepted x means that if v is in an intact set, it could never be in a quorum for a value other than x. Finally, if v is in a quorum in which every node has accepted x, v *confirms* x and locally outputs x as the decision of the vote.

3.2 Externalize messages

Though the specifics of the FBA consensus protocol do not affect bridge design, once a node chooses a block through consensus, it generally digitally signs an *externalize* message stating that it has committed the block. Externalize messages are not strictly required for normal operation, but help nodes that have fallen behind catch up. An externalize message signed by node v for block b means that not only did v reach the accept phase of whatever federated vote was required to choose b, but v also verified that every other member of a quorum accepted b. Later on, it could be that some members of that quorum left the system and no longer exist, so it is handy not to have to worry about them. Where ordinarily you need to check a quorum of messages, you only need to check a quorum *slice* of externalize messages.

Light clients primarily ingest externalize messages from a foreign blockchain. These are particularly useful to bridges for two reasons. First, it is simpler to verify foreign quorum slices than foreign quorums. Second, because externalize messages are not required by the FBA protocol, they provide an opportunity to piggyback additional assertions in a digitally signed message. In particular, we can modify the semantics of externalize messages to include additional assertions about foreign blockchains.

4 Informal protocol

The job of a light client for blockchain C, embedded in validators of blockchain B, is to determine which block headers are *valid* on C and to maintain a growing sequence of *imported* block headers from C. Each imported block header in the sequence is the parent of the next block header, which (barring a reorg of C) means that the child contains a collision-resistant hash of the parent.

To recover from a reorg of C, B's light clients may hard-code a few pairs of the form $\langle c_P, c\rangle$ in a *splice table*. This states that c_P is the parent of c, even if c's parent pointer does not match a hash of c_P. Moreover, it makes c_P inherently valid and imported, without regard to messages from other nodes. The rest of this section explains the normal case of how blockchain B imports and validates block headers from blockchain C in the absence of splices.

If v is a B-validator (i.e., a validator for blockchain B), a block header c of blockchain C is *valid at* v if v's light client has seen externalize messages for c from one of its quorum slices for C. We assume that light clients in B-validators monitor and multicast C's externalize messages among themselves. Hence, if a non-faulty light client in a B-validator sees an externalize message from a C-validator, eventually all light clients in non-faulty B-validators will see the same message. Nodes disseminate externalize messages regardless of consistency. If a faulty C-validator equivocates, signs two contradictory externalize statements, and sends them to two different non-faulty B-validators, eventually both B-validators will see both externalize messages. This means multiple foreign block headers with the same parent can both be valid at the same B-validator.

Light clients in B-validators *import* block headers from C by conducting a federated vote on statements of the form:

$$\textsc{import}\ C\ c_P\ c.$$

A B-validator v can only vote for such a statement if c is valid at v and c_P is c's parent. In addition, v can only vote for one child c for a given parent c_P, and can only accept one value of c for a given c_P. However, following the rules of federated voting, v can accept an IMPORT statement without voting for it (and hence without yet believing it valid) if a v-blocking set accepts the statement. When v confirms $\textsc{import}\ C\ c_P\ c$, then c is considered imported by v.

In the absence of bridges, the consensus values decided by FBA blockchains are actions of the form, "Append block header b to blockchain B." With bridges, the consensus values are actions conditioned on the validity of foreign block headers, which can be read as, "Append block header b

(which references foreign block header c) to blockchain B *if c is a valid block header for blockchain C*." Consensus leaders should set c to the latest imported block header from C in block proposals. Block validators should accept any imported c that is no older than the c in the previous block.

To make externalize messages actionable, we modify their semantics to remove any caveats about foreign blockchains. In other words, after consensus at B-validator v decides that b is valid if c is valid, v cannot sign an externalize message for b or expose b to client-facing services until c is valid at v.

A few informal observations motivate this design. Suppose blockchain B has an intact quorum in which every validator's light client has the exact same set of quorum slices for blockchain C. Then no failure of C-validators can impact B's liveness, because all intact B-validators will agree on which blocks of C are valid, so totality of the IMPORT vote ensures eventual agreement on any valid c.

Suppose, alternatively, that B never suffers a node failure, but that its light-client configurations are an incoherent mess and that blockchain C forks. Once again, B will keep producing blocks, as any foreign block header c proposed to FBA consensus must have received a quorum of IMPORT votes. There is one subtlety, however: a well-behaved B-validator v could stop externalizing blocks if a quorum of validators with different light client configurations imports a block that v considers invalid. Whereas v would be intact without the bridge, its subjectively "faulty" (in the eyes of a quorum) choice of quorum slices for C can cost v liveness.

A bridge can also prevent all B-validators from externalizing new blocks in the event that blockchain C forks, there is genuine disagreement between B-validators as to which branch of the fork constitutes the "real" blockchain C, and some B-validators also fail. Suppose a "c-valid" faction of B-validators believes c is the next block header for C, while a "c-invalid" faction believes that either $c' \neq c$ is the next valid block header, or there is no later valid header. If neither faction contains a quorum, B will continue to be live, as no further IMPORT votes can succeed for C. On the other hand, if the c-valid faction contains a quorum, it can continue to commit new blocks to B referencing c and c's descendants, while the c-invalid faction eschews these new blocks. If, at some point, some B-validators in the c-valid faction fail, B can lose liveness, even though, in the absence of a bridge, there would be an intact quorum spanning the two factions.

An analogous liveness issue arises with protocol upgrades: a quorum of B can vote to upgrade the software to a version of the protocol that is not supported by all validators. If some of the nodes that voted to upgrade subsequently fail, the network can lose liveness even when there would be a quorum at the old protocol version. The solution is to upgrade the remaining nodes to support the new protocol. Similarly, in the bridge scenario, a solution to regain liveness is to synchronize foreign quorum slices across light clients. One of the benefits of FBA consensus is that nodes can change their quorum slices at any time, with no need to agree on the reconfiguration through consensus.

5 Formalization

In this section, we formalize the proposed bridge architecture and show its exact security properties. Section 5.1 introduces nodes and how they communicate. Section 5.2 introduces validators for different blockchains and their FBA configurations. Section 5.3 introduces the core blockchain datastructures, block headers and blocks, and defines application validity. Section 5.4 specifies the guarantees of a bridged FBA blockchain, and Section 5.5 shows under which conditions the liveness guarantees of mutually bridged blockchains compose. Section 5.6 presents an architecture and implementation that provide these guarantees. Section 5.7 notes that, with mutual bridges, externalizing decided blocks can take unboundedly long, and sketches a remedy. We defer splices to Section 6.

5.1 Network model

We consider a message-passing distributed system consisting of a finite set of nodes N. Each node in N faithfully runs the protocols that it is assigned, except for an unknown, fixed subset F of faulty nodes, which may deviate arbitrarily from any protocol they are assigned. We say that the other nodes are well-behaved and we define the set of well-behaved nodes $W = N \setminus F$.

We assume that each node may broadcast messages and that there is a constant Δ, unknown to the nodes, such that every message broadcast by a well-behaved node is received by all well-behaved nodes within Δ time. This is a simpler variant of the traditional eventual-synchrony assumption, where the bound Δ holds only after an unknown time called the global stabilization time (GST). We assume that each node has a unique public/private key pair allowing it to authenticate the messages it sends by signing them, and that faulty nodes cannot forge signatures. Finally, we assume that only finitely many events occur in any bounded period of time.

5.2 FBA configurations

We assume that each node in the system may be a validator of at most one blockchain among a finite set of blockchains $\mathcal{B}$. By convention, when a notion is relative to a blockchain B, we may use the prefix "B-" to indicate this. For example, we may say that a node is a B-validator to mean that it is a validator of blockchain B. We write V_B for the set of validators of a blockchain $B \in \mathcal{B}$ and assume that each validator $v \in V_B$ has a non-empty set $\mathcal{S}_B(v)$ of non-empty subsets of V_B called quorum slices. This determines an FBA configuration $\Sigma_B = (V_B, \mathcal{S}_B)$.

In general, an FBA configuration Σ is a pair $(V, \mathcal{S})$ where $V \subseteq N$ is a set of validator nodes, and, for every validator $v \in V$, $\mathcal{S}(v)$ is a non-empty set of sets of validators called v's quorum slices. Now consider an FBA configuration $(V, \mathcal{S})$.

We say that a set Q of validators is a quorum when every member of Q has a quorum slice included in Q. Moreover, Q is a quorum of a validator v when it additionally includes a slice of v. A direct consequence is that a quorum is a quorum of all its members. As defined in Section 3.1, a set of validators V' blocks a validator v, or is v-blocking, if every slice of v intersects V'.

For the purpose of solving agreement problems among a set of nodes, it is important that two quorums of members of the set have non-faulty nodes in common (to avoid disagreement) and that each member of the set have a non-faulty quorum (to make progress), and this despite faulty nodes potentially lying about their slices. We next introduce the notion of intertwined and intact sets to formalize these requirements.

First, to account for the potential effect of faulty nodes on slices and quorums, we introduce the notion of projection on a set and deletion of a set of nodes. Given a set of validators $V' \subseteq V$, the projection $\Sigma \restriction V'$ removes all validators not in V', truncating each slice to V'. We define it as $\Sigma \restriction V' = (V', \mathcal{S}')$ where, for every validator $v \in V'$, $\mathcal{S}'(v) = \{S \cap V' \mid S \in \mathcal{S}(v)\}$. Conversely, given a set of nodes X, deleting X from Σ yields $\Sigma \ominus X = \Sigma \restriction (V \setminus X)$.

Note that both quorum slices and quorums may be empty after a projection, and so our definitions above allow empty quorum slices and empty quorums. However, validators in a real network should not choose empty quorum slices.

We say that a set of validators V' has quorum intersection despite another set X of validators when, in $\Sigma \ominus X$, every two quorums of members of V' intersect. Moreover, V' has quorum availability despite X when, in Σ, every member of V' has a quorum disjoint from X.

We say that a set of validators V' is intertwined when V' is non-empty and well-behaved, and V' has quorum intersection despite the set of faulty validators F. Two validators v and v' are intertwined when $\{v, v'\}$ is intertwined. We say that a set of validators I is intact when I is non-empty and well-behaved, I is a quorum, and I has quorum intersection despite its complement.

We say that a set is intact in blockchain B, or B-intact, when it is intact in Σ_B, and similarly for intertwined. Note that intact sets are sets that are self-sufficient in the sense that they have quorum intersection and availability despite their complement.

It is easy to see that every intact set is intertwined. We can also prove that the union of two intersecting intact sets is an intact set; since the notion of agreeing on something is inherently transitive, this makes sense if members of intact sets are to agree with each other. Finally, intact sets satisfy the cascade theorem:

Theorem 5.1 (Cascade). *Let I be an intact set, Q a quorum that intersects I, and S a set that includes Q. If $I \not\subseteq S$, then some member of $I \setminus S$ is blocked by $I \cap S$.*

Thus, starting from $S = Q$ and repeatedly adding to S the members of I that $I \cap S$ blocks eventually covers I. This guarantees that once the members of I in a quorum agree on something, the agreement cascades to all of I. Section A gives the proof.

5.3 Block Headers and Blocks

The core functionality of a blockchain is to agree on a growing sequence of blocks. Validators let the outside world (users, through wallets, block explorers, etc.) know the sequence of blocks they agreed upon by externalizing block headers.

A block header b is a data structure associated with a block, denoted $\mathsf{block}(b)$, and an optional parent, denoted $\mathsf{parent}(b)$, which is another block header. Block headers generally specify their block and parent using collision-resistant cryptographic hash values (using a special sentinel value $\perp$ for no parent).

We assume that each blockchain B has bridges from a fixed set $\mathsf{foreign}(B) \subseteq \mathcal{B}\setminus\{B\}$ of blockchains called B's foreign blockchains. Typically, bridges are bi-directional (i.e., $C \in \mathsf{foreign}(B)$ if and only if $B \in \mathsf{foreign}(C)$), but this is not required. In practice, the set of foreign blockchains of B can be changed during a configuration upgrade, but this is outside the model.

In a bridged blockchain B, a block comprises a transaction set and a set of foreign block headers, each uniquely associated to a foreign blockchain in $\mathsf{foreign}(B)$, with at most one foreign header per foreign blockchain. The foreign block headers correspond to the latest known state of each foreign blockchain as of when the block was assembled. We write $\mathsf{fh}(b)$ for the set of foreign block headers appearing in the block associated with header b. For simplicity, we often refer to components or properties of a header's block as the header's components or properties (e.g., a header's transaction set or C-header, when we mean those of the corresponding block).

We say that a block header b' is an ancestor of a block header b and, equivalently, that b is a descendant of b', when $b' = b$ or b' is reachable from b by following the parent relation. We assume that the parent relation is acyclic and well founded (meaning that every block has a finite chain of ancestors). We say that two block headers b and b' are compatible when one is an ancestor of the other, and incompatible otherwise.

Each blockchain B has a fixed, trusted genesis header g_B, known to all its validators, which has no parent and no foreign headers. Its block specifies the initial application state.

A block's execution on top of its ancestry determines a set of events that describe its effects (e.g., account A_1 transferred 10 XLM to account A_2). Depending on the blockchain, the events associated with a block may be included in the block or, to allow pipelining execution, in the next block.

A cross-chain transaction involving a foreign blockchain C depends on the state of C, and it includes a validity proof. For example, to transfer tokens from C to B, users submit transactions on B that call a bridge smart contract and provide a proof that a matching deposit on C was made. Such a proof might consist of a C-header c' along with a Merkle path showing that the matching deposit is among the events emitted by the execution of c' on C. We say that such a transaction,

appearing in a block with C-header c, is valid with respect to c when its validity proof is correct, c' is an ancestor of c, and, in the chain determined by the block, the corresponding events have not yet been consumed at the point where the transaction appears.

We say that a B-header b is application-valid when the following holds:

(1) Header b and each of its ancestors and their blocks are well-formed (e.g., conform to a particular XDR schema),
(2) Foreign references are monotonic: if b'' is an ancestor of b' and both are ancestors of b, then, for each foreign blockchain C, if b'' has a C-header, then so does b', and the C-header of b'' is an ancestor of that of b'.
(3) Executing the transaction sets of b's ancestry, each on top of its parent, is well defined, where executing a cross-chain transaction involving a foreign blockchain C includes checking that it is valid with respect to the block's C-header.

Monotonicity ensures that, along a chain of B, references to each foreign blockchain never go backward or switch forks. Since these checks use only the headers involved and the parent relation on foreign headers, which is a property of the headers themselves, whether b is application-valid does not depend on the validator checking it.

5.4 Bridged FBA Blockchains and their Guarantees

A bridged FBA blockchain B is a distributed protocol run by the set of B-validators V_B. Validators interact with users of the blockchain by receiving user transactions and repeatedly creating and externalizing new block headers. Externalizing a block header by convention also externalizes its ancestors and the corresponding blocks, and the genesis block is externalized from the start. Upon externalizing a block, a validator broadcasts an externalize message on the network containing the externalized block header, and a well-behaved validator sends no externalize message for a header that it has not externalized. We assume that the corresponding block and its events are reliably disseminated throughout the system by some unspecified process.

5.4.1 User-facing safety properties. Users typically expect a blockchain to guarantee two safety properties: First, externalized blocks are application-valid, e.g., they are well-formed and their transactions' execution, on top of their ancestry, is well defined. Second, externalized blocks, even if externalized by different validators, form a single chain; this is often called the consistency property. In an FBA blockchain, which validator is guaranteed to remain consistent with which other depends on the FBA configuration: only intertwined validators are guaranteed consistency.

In a bridged blockchain, application validity checks cross-chain transactions against the foreign headers embedded in native blocks (Section 5.3), but not the foreign headers themselves. Ideally, each C-header c appearing in an externalized B-header should moreover be valid in C and have been externalized by C. Unlike application validity, this depends on what C externalizes. For each foreign blockchain C, each validator v of B therefore embeds a light client of C that observes C's externalizations. However, whether C as a whole externalized a header is not well-defined unless all its validators are intertwined, which cannot be guaranteed. Instead, the light client of C on v is configured with its own non-empty set $\mathcal{S}_B^C(v)$ of C-slices, each a non-empty set of validators of C. We say that a C-header c is valid at v when v's light client for C has received an externalize message externalizing c (that is, for c or a descendant of c) from each member of one of its C-slices. We also say that such a set of externalize messages forms a certificate for c at v. By default, the genesis header g_C is valid at every B-validator (in practice, g_C can be given as configuration parameter of light clients).

A B-header b is then valid at a validator v when it is application-valid and every foreign header of b and of its ancestors is valid at v.

We are now ready to state the two safety properties of bridged FBA blockchains precisely:

- ***Validity.*** If a well-behaved B-validator v externalizes a header b, then b is valid at v when v externalizes it.
- ***Consistency.*** A well-behaved B-validator never externalizes incompatible headers, and two intertwined B-validators never externalize incompatible headers.

Together with the monotonicity of foreign headers, *Validity* and *Consistency* imply that externalized foreign headers are consistent too: for every foreign blockchain C, if a well-behaved B-validator, or two intertwined ones, externalize b containing c and b' containing c', then c and c' are compatible.

Importantly, note that *Validity* and *Consistency* make no assumptions on the behavior of foreign blockchains. Thus, foreign blockchains cannot cause safety violations on the native blockchain.

5.4.2 User-facing liveness properties. Users also expect a blockchain to make progress: validators should keep externalizing new blocks, and these blocks should keep including the latest state of foreign blockchains. In an FBA blockchain, even without bridges, only intact sets of validators are guaranteed to make progress.

In a bridged blockchain, progress additionally depends on the view that light clients have of foreign blockchains: the *Validity* property requires that the foreign headers of the headers that a validator v externalizes be valid at v, and that depends on the externalize messages from other blockchains received by v's light clients and their C-slices. Moreover, since light clients on different B-validators may use different C-slices, a C-header valid at one B-validator need not ever become valid at another, even if they receive the same foreign externalize messages. This means that a B-validator in an intact set I could externalize a header b with a foreign header c while other members of I are unable to determine that c is valid and thus never externalize b.

Because we want to err on the side of safety and maintain strong validity, *Liveness* and *Foreign Liveness* below assume that foreign-header validity is *total* in an intact set. We say that foreign-header validity is total in a set of B-validators V' when, once the light client of a member v of V' determines that a foreign header is valid at v, all members of V' eventually make the same determination. This definition imposes no bound on the delay (see Section 5.7).

Users also expect externalized blocks to include their transactions, but which transactions a block can include depends on the blockchain's capacity limits, fee market, etc., which we do not want to fix. We therefore parameterize B by an inclusion policy π_B, which is a predicate $\pi_B(b, R)$ on a B-header b and a finite set R of transactions. Intuitively, $\pi_B(b, R)$ means that the transaction set of b is an acceptable choice, on top of b's parent, for a validator that has received the transactions in R, i.e., that b does not neglect R. Typically, $\pi_B(b, R)$ holds trivially once the transactions of R appear in ancestors of b or can no longer be included, e.g., because they conflict with included ones. We require that π_B be antitone in R, i.e., if $R \subseteq R'$, then $\pi_B(b, R')$ implies $\pi_B(b, R)$, and that it can always be met: any parent and foreign headers that admit an application-valid header b admit one with $\pi_B(b, R)$.

For example, the trivial policy that always holds allows empty blocks. Without capacity limits, a policy may require b to be maximal for R: no nonempty set of transactions of R can be added to the transaction set of b while keeping b application-valid. With fees and capacity limits, it may instead allow b to omit such transactions only if b is full and each of its transactions pays at least as much as each omitted one.

Armed with these notions, we can now state the liveness properties that a bridged FBA blockchain B should guarantee:

- ***Liveness.*** For every B-intact set I in which foreign-header validity is total, and every finite set R of transactions that every member of I eventually receives, every member of I externalizes infinitely many headers b such that $\pi_B(b, R)$.
- ***Foreign Liveness.*** For I as in *Liveness*, for every foreign blockchain C, let b be a header externalized by a member of I, let c_b be the C-header of b (or the genesis g_C if b has none), and let c be a descendant of c_b. Suppose c becomes valid at a member of I, and no descendant of c_b incompatible with c is ever valid at a member of I. Then every member of I eventually externalizes a B-header whose C-header is a descendant of c.

Taking $R = \emptyset$, *Liveness* implies that every member of I externalizes infinitely many headers. With the fee-based policy above and $R = \{t\}$, *Liveness* implies that, for a transaction t received by every member of I, either t is eventually included, or infinitely many externalized headers cannot accept t or are full of transactions paying at least as much as t. Also note that, because foreign blockchains might fork, *Foreign Liveness* is stated relative to an externalized header b and only applies to foreign descendants of b's foreign header that do not get contradicted.

Note that, unlike the safety properties of the preceding subsection, in general *Liveness* and *Foreign Liveness* depend on the behavior of foreign blockchains. However, one special case allows liveness isolation from foreign chains: If all members of an intact set I have exactly the same foreign slices and light clients forward externalize messages to each other, then, foreign-header validity is total in I regardless of the behavior of foreign blockchains; thus, the guarantees of *Liveness* and *Foreign Liveness* become independent of the behavior of foreign blockchains. Theorem B.1 in Section B formalizes this property.

Finally, we want to ensure that, when total foreign-header validity does not hold (say because a foreign chain forked), if a particular externalized header becomes valid at all members of an intact set and some member externalizes it, then all eventually do. Formally, we require the following additional property:

- ***Externalization Totality.*** For every B-intact set I, if a member of I externalizes a header b, and b eventually becomes valid at every member of I, then every member of I eventually externalizes b.

This property is also instrumental in breaking the circularity of the liveness assumptions between two mutually bridged blockchains when proving Lemma 5.3.

5.5 The Liveness Composition Theorem

Throughout this subsection, we assume that each blockchain satisfies the properties of Section 5.4. Theorem 5.7 establishes these properties for our implementation.

As we have seen in the previous section, *Liveness* and *Foreign Liveness* of an intact set are conditional on foreign-header validity being total in the intact set and, in general, this depends on the behavior of foreign blockchains. In this section, we define mutually aligned intact sets, a condition relating intact sets of different blockchains (which depends on their FBA configurations and on which validators are faulty), and show that mutually aligned intact sets enjoy an unconditional version of *Liveness* and *Foreign Liveness*.

For simplicity, let us first consider a single foreign blockchain C with a bi-directional bridge to our native blockchain B. The crux is to ensure that foreign-header validity be total. We have seen that this is easy to achieve in B, independently of C, when the foreign slices of the validators of B are identical (see the discussion in the previous section, and Theorem B.1). But what if they are not? Suppose that a member v of a B-intact set I_B externalizes a header b containing a C-header c, having received externalize messages for c from the members of one of its C-slices. Another member v' of I_B may have different C-slices, each containing a C-validator that never externalizes c, e.g.,

because it is faulty or not intertwined with the C-validators that externalized c. Then v' never obtains a certificate for c, so it never externalizes b or any of its descendants and, since v and v' are intertwined, it cannot externalize headers incompatible with b either: v' loses liveness.

Beyond all foreign slices being the same, are there more relaxed conditions that ensure this cannot happen? A complication is that, with mutual bridges, whether foreign-header validity is total in I_B depends on externalization in C, which depends in turn on which B-headers are valid at C-validators. As we now show, mutually aligned intact sets of B and C provide each other's totality requirements and achieve unconditional liveness.

Let Σ_B^C be the FBA configuration of C, Σ_C, in which we add B's light clients as virtual validators, i.e. $\Sigma_B^C = (V_B \cup V_C, \mathcal{S}_B^C \cup \mathcal{S}_C)$. Here, $\mathcal{S}_B^C \cup \mathcal{S}_C$ is simply the two functions' pointwise union, which is well-defined since their domains are disjoint. We say that a set V' of B-validators is C-aligned with a set I_C of C-validators when $V' \cup I_C$ is intact in Σ_B^C. Moreover, we say that a B-intact set I_B and a C-intact set I_C are mutually aligned when I_B is C-aligned with I_C and I_C is B-aligned with I_B. The following characterization gives the two slice conditions used in the proofs below.

Lemma 5.2 (Alignment). *A set V' of B-validators is C-aligned with a set I_C of C-validators if and only if V' is well-behaved, I_C is C-intact, and every member v of V' satisfies:*

(1) *Every C-slice of v intersects I_C.*
(2) *Some C-slice of v is included in I_C.*

Section A gives the proof.

It may seem that, with mutual bridges, establishing the liveness of I_B requires the liveness of I_C, and vice versa. However, the following lemma relies on no liveness property: it uses only *Validity* and *Externalization Totality*, which make no assumptions on foreign blockchains, and proceeds by induction on the causal order of externalizations.

Lemma 5.3. *Suppose that B and C are each other's only foreign blockchains, and that a B-intact set I_B and a C-intact set I_C are mutually aligned. Then every B-header externalized by a member of I_B is eventually externalized by every member of I_B, and foreign-header validity is total in I_B. The same holds for C and I_C.*

Section A gives the proof. In turn, the lemma implies our composition theorem:

Theorem 5.4 (Liveness composition). *Under the hypotheses of Lemma 5.3:*

(1) *The guarantees of Liveness and Foreign Liveness apply to I_B and to I_C.*
(2) *If a member of I_C externalizes a C-header c, then every member of I_B eventually externalizes a B-header whose C-header is a descendant of c. The same holds with B and C exchanged.*

The second part is stronger than *Foreign Liveness* alone: *Foreign Liveness*'s hypothesis that no descendant of c_b incompatible with c is ever valid at a member of I_B is discharged by alignment and the *Consistency* of C, since every C-header valid at a member of I_B is g_C or was externalized by a member of I_C. Section A gives the proof.

Lemma 5.3 and Theorem 5.4 generalize to any number of blockchains with any bridges between them, provided that each blockchain X has an X-intact set I_X that is Y-aligned with I_Y for each foreign blockchain Y of X.

Finally, note that while Lemma 5.3 shows that foreign-header validity is total for mutually aligned intact sets, it does not provide any real-time delay bound. Unfortunately, in some configurations, this delay may grow unboundedly and postpone externalization proportionally. Section 5.7 addresses this issue in detail.

5.6 Architecture and Implementation

We now describe how to implement the guarantees of Section 5.4 by reusing a partially-synchronous FBA consensus protocol such as SCP. Figure 2 depicts the software stack on a validator. It consists of four main components: application, light clients, block manager, and consensus layer (Figure 2 depicts only one light client, but in general there is one per foreign blockchain). The application is typically a replicated ledger supporting smart contracts, and it is what users of the blockchain interact with. The consensus layer agrees on, or decides, B-headers; decided B-headers may however not be ready for externalization yet. The block manager supplies consensus with new headers to propose for decision in the consensus protocol and tells it which headers it may vote for or decide (their admissibility). It passes decided headers to the application and externalizes them once their foreign headers are valid. For each foreign blockchain C, the light client for C uses C's externalize messages to determine which C-headers are valid at the validator, and imports C-headers by federated voting among B-validators on valid C-headers (Section 4).

The main constraint is that partially-synchronous consensus protocols need timely admissibility for liveness: a header eligible for voting at one member of an intact set must become eligible at every member within a fixed delay. Even with mutually aligned intact sets, foreign-header validity need not satisfy this constraint. We therefore import foreign headers through federated voting; externalization additionally requires them to be valid.

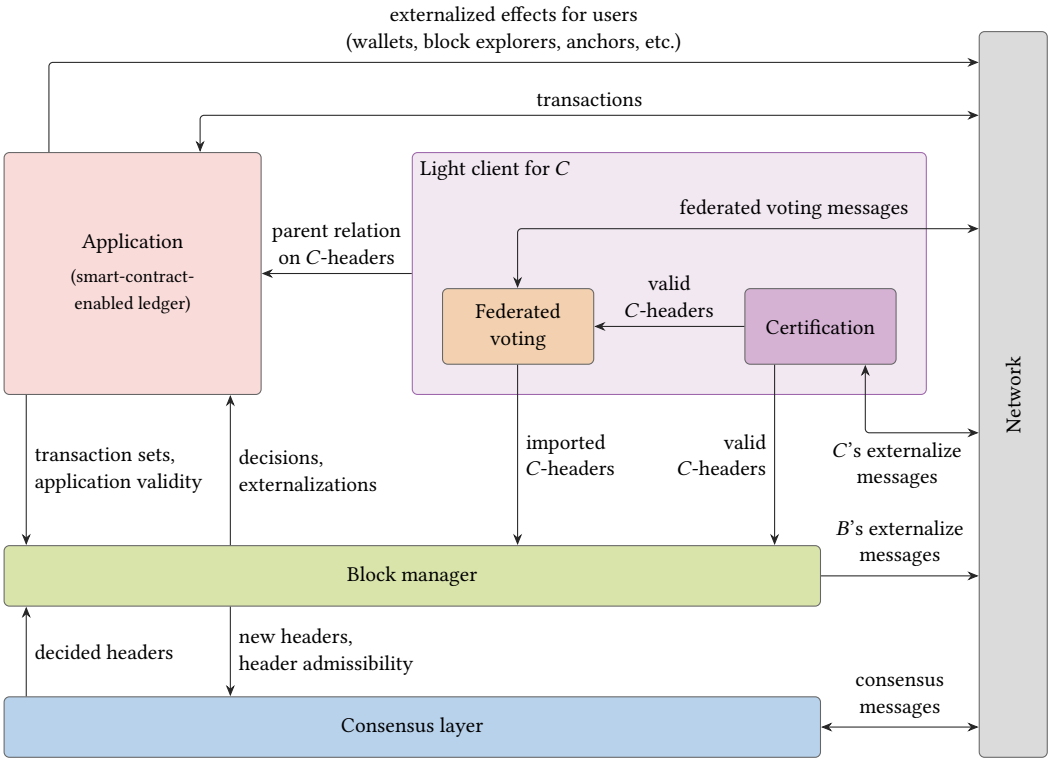


Fig. 2. Component stack at each B-validator, for a single foreign blockchain C; with several foreign blockchains, each has its own light client. Arrows show the flow of information; arrows to and from the network show messages sent and received.

5.6.1 Application interface. At each well-behaved B-validator, the application starts with the trusted genesis header g_B decided and externalized. B interacts with the application through four procedures:

- decide(b) notifies the application that header b, and by convention each of its ancestors, is decided. A decided header is final and new blocks may build on it, but the application must not act on it, e.g., by reporting the effects of its transactions to users, until it is externalized.
- externalize(b) notifies the application that the decided header b, and by convention each of its ancestors, is externalized. Externalizing a header b also implies broadcasting the externalize message EXTERNALIZE B b (Section 5.4).
- get_txset(b, H) requests a transaction set for a new block with parent b and foreign headers H, where H contains exactly one C-header for each $C \in \mathsf{foreign}(B)$.
- is_txset_valid(b) asks the application whether the transaction set in b is valid with respect to b's parent and foreign headers.

Since the blockchain may decide a block well before externalizing it, the application must validate and build on blocks whose parents are decided but not yet externalized.

We assume:

- The result of is_txset_valid(b) depends only on b and its ancestors. Moreover, if parent(b) is application-valid (Section 5.3), then is_txset_valid(b) holds if and only if b is well-formed and executing its transaction set on top of its ancestry is well defined, including the checks of cross-chain transactions against the foreign headers of b.
- If b_P is application-valid and each C-header in H descends from the C-header of b_P, whenever b_P has one, then get_txset(b_P, H) eventually returns a transaction set T such that the header b of the block with parent b_P, foreign headers H, and transaction set T is application-valid and satisfies $\pi_B(b, R)$, where R is the set of transactions that the calling validator has received when the call begins.

The trusted genesis header g_B is application-valid. Since monotonicity of foreign references holds along a chain as soon as it holds between each header and its parent, a non-genesis header b is thus application-valid if and only if parent(b) is application-valid, is_txset_valid(b) holds, and, for each foreign blockchain C such that parent(b) has a C-header, b has a C-header that descends from it. The block manager checks the last condition using the light clients (Section 5.6.2); it holds trivially when parent(b) = g_B, which has no foreign headers.

5.6.2 Light clients. At a B-validator v, the light client for each foreign blockchain C has a certification component that determines which C-headers are valid at v, using the C-slices and certificates of Section 5.4. It rebroadcasts the externalize messages that it receives from C's validators, so that every well-behaved validator eventually holds the same messages. A certificate for c may combine messages for c and for different descendants of c. Since externalizing a header externalizes its ancestors, validity at v is closed under ancestry.

The set of C-headers valid at v only grows. Valid headers need not be compatible: for example, if C's validators equivocate, conflicting C-headers may both be valid at v.

To not tie B too strongly to C's header format and to the way C handles splices (Section 6), the application obtains C's parent relation from the light client for C by asking is_parent(c_P, c), which, without splices, holds when c_P = parent(c). The light client's answers thus agree with the parent relation used by application validity (Section 5.3) and are the same at every validator. This lets smart contracts ask whether a C-header c is valid: a contract executing in a block with C-header c', determined by H in get_txset(b_P, H) or by fh(b), considers c valid when c is an ancestor of c',

which it checks using is_parent. By *Validity* in Section 5.4, c' is valid at v once v externalizes the block, so c is too.

Each light client starts with the genesis C-header g_C imported. Since g_C is valid by definition, the first new B-block can reference it without waiting for messages from C. The federated-voting component runs federated voting (Section 3.1) among B-validators, with their slices in Σ_B, on statements IMPORT C c_P c (Section 4), where $c \neq g_C$ is a C-header and $c_P = \text{parent}(c)$. We say that two such statements conflict when they have the same c_P and different c. A well-behaved v votes for at most one, and accepts at most one, of any two conflicting statements. It votes for IMPORT C c_P c once c is valid at v, unless it has already voted for or accepted a conflicting statement, and never before c is valid at v. It accepts IMPORT C c_P c following the rules of federated voting, however, whether or not c is valid at v.

We say that a non-genesis header c is imported at v when v confirms IMPORT C c_P c and c_P is an imported C-header at v. Since v accepts at most one child of each C-header, the C-headers imported at v form a chain starting at g_C, whose last element is v's latest imported C-header. If C's validators equivocate, the votes of B-validators may split between two children of a C-header, and B may then never import beyond it; *Foreign Liveness* excludes this case by hypothesis.

LEMMA 5.5. *Let I be a B-intact set and c a C-header imported at a member of I at time t. Then c is valid at a member of I, and c is imported at every member of I by time $t + |I|\Delta$.*

The lemma holds for g_C by initialization. For other headers, the first part holds because the first member of I to accept an IMPORT statement must have voted for it. The second follows from the cascade theorem (Theorem 5.1), since members of I accept IMPORT statements through blocking sets whether or not the header is valid at them. Section A gives the proof.

5.6.3 Consensus. At each well-behaved B-validator, consensus starts with the genesis header g_B decided. We say that a B-header b is admissible at a validator v once v has obtained the data needed to check b's application validity, checked that b is application-valid, and imported each foreign header of b and of its ancestors. The trusted genesis header g_B is admissible from the start. Once admissible at v, b stays admissible at v. The block manager supplies the consensus layer with new headers to propose and with their admissibility, and the consensus layer of a well-behaved v votes only for headers admissible at v.

The data needed to check a header's application validity include its ancestors and their blocks and any foreign-header ancestry data needed to check cross-chain transactions; they do not include certificates establishing foreign-header validity at the validator. We assume that, once a well-behaved validator has obtained a header and these data, it completes the check, including any unchecked ancestors, within a fixed time Δ_{val}. This bound does not cover obtaining these data, which a faulty proposer may withhold. What matters is propagation within an intact set: the block manager relays the data with every header that becomes admissible (Section 5.6.4), so they reach every well-behaved validator within Δ of the header first becoming admissible at a well-behaved one.

We say that admissibility is timely in a set V' of B-validators when there is a fixed bound D such that, if a B-header is admissible at a member of V' at time t, then it is admissible at every member of V' by time $t + D$. We assume that the consensus layer guarantees the following, for every B-intact set I:

- ***Consensus Validity.*** If the consensus layer of a member of I decides a header, then the header is admissible at a member of I.

- ***Consensus Consistency.*** The consensus layer of a well-behaved B-validator never decides incompatible headers, and those of two intertwined B-validators never decide incompatible headers.
- ***Consensus Totality.*** If the consensus layer of a member of I decides a header, then those of all members of I eventually decide it.
- ***Consensus Liveness.*** If admissibility is timely in I, then the consensus layers of members of I decide infinitely many headers proposed by members of I.

These are the usual guarantees of consensus protocols for partial synchrony, adapted to the FBA model [9].[1]

Consensus Liveness needs a fixed bound because consensus relies on timeouts, which must eventually exceed the time that members of I take to find each other's proposals admissible; like Δ, D need not be known. Since application validity does not depend on the validator (Section 5.3), the data relay and the bound Δ_{val} above, together with Lemma 5.5, imply:

Lemma 5.6. *Admissibility is timely in every B-intact set I, with $D = |I|\Delta + \Delta_{val}$.*

Section A gives the proof. The bound D depends neither on C nor on B's C-slices, whereas foreign-header validity need not be timely in I, even under mutual alignment (Section 5.7).

Requiring only that headers be application-valid would also be timely, but too weak: a faulty proposer could then get members of I to decide a block whose C-header is never valid at any member of I, and no member of I could externalize that block, or any later block, since later blocks descend from it. Requiring imports through federated voting rules this out, because a header imported at a member of I is valid at a member of I (Lemma 5.5).

5.6.4 Block manager. The block manager of a well-behaved B-validator v follows these rules:

- To construct a new header to propose, v chooses, according to the consensus protocol, a parent b_P admissible at v and makes a new call to get_txset(b_P, H), where H contains v's latest imported C-header for each foreign blockchain C when the call begins. Each call constructs one header, containing exactly one header per foreign blockchain; Section 5.3 allows at most one. Since b_P is admissible at v, its foreign headers, if any, are imported and hence ancestors of the corresponding headers in H. The assumption on get_txset therefore makes the new header application-valid. Its foreign headers and those of its ancestors are all imported at v, so it becomes admissible once v checks its application validity. This also covers $b_P = g_B$, which has no foreign headers.
- When a header becomes admissible at v, v broadcasts it together with all data needed to check its application validity (Section 5.6.3). These data are already available at v, and all well-behaved B-validators receive them within Δ.
- v calls decide(b) once its consensus layer decides b and b is admissible at v, and externalize(b) once, moreover, b is valid at v.

5.6.5 Correctness.

Theorem 5.7. *If the application satisfies the assumptions of Section 5.6.1, the consensus layer provides the guarantees of Section 5.6.3, and well-behaved B-validators follow the rules of Section 5.6, then B satisfies the properties of Section 5.4.*

[1] SCP's liveness additionally assumes periods without new messages from faulty nodes; otherwise, faulty nodes can perpetually preempt progress. Since this is not inherent to the FBA model, we assume the stronger *Consensus Liveness* property stated above. A weaker liveness property, such as SCP's, could also be used with timely admissibility; our *Liveness* and *Foreign Liveness* guarantees would then inherit its additional assumptions.

Validity follows from the rules, and *Consistency* from *Consensus Consistency*. For the other properties, consider a B-intact set I. Since admissibility is timely in I (Lemma 5.6), by *Consensus Liveness*, consensus keeps deciding headers, and, by Lemma 5.5, their foreign headers are valid at a member of I; if foreign-header validity is total in I, these foreign headers thus become valid at every member of I, which can then externalize the headers. Section A gives the proof.

The implementation also guarantees safety for decided headers, before they are externalized. Every header decided by a well-behaved validator is application-valid, because the block manager requires admissibility before calling decide. By *Consensus Consistency*, a well-behaved validator never decides incompatible headers, and neither do two intertwined validators. Together with application validity's monotonic foreign references, this implies that the C-headers of their decided blocks are compatible too. What decided headers miss is validity of foreign references.

5.7 Mutual Bridges and Unbounded Delays

The following example shows that foreign-header validity can be total without being timely, even under mutual alignment. Suppose B and C each have three validators, v_1, v_2, v_3 and v'_1, v'_2, v'_3. On B, v_1 has local slices $\{v_1, v_2\}$ and $\{v_1, v_3\}$ and a single foreign slice $\{v'_1\}$, v_2 has a single local slice $\{v_1, v_2\}$ and a single foreign slice $\{v'_2\}$, and v_3 is faulty; the situation is symmetric on C (Table 1). Then $\{v_1, v_2\}$ and $\{v'_1, v'_2\}$ are intact and mutually aligned. Suppose that messages among v_1, v_3, v'_1, and v'_3 are instantaneous, while messages to or from v_2 or v'_2 take Δ.

Table 1. Configuration of the mutual-bridge example.

Validator	Blockchain	Local slices	Foreign slice
v_1	B	$\{v_1, v_2\}$, $\{v_1, v_3\}$	$\{v'_1\}$
v_2	B	$\{v_1, v_2\}$	$\{v'_2\}$
v_3	B	Faulty	
v'_1	C	$\{v'_1, v'_2\}$, $\{v'_1, v'_3\}$	$\{v_1\}$
v'_2	C	$\{v'_1, v'_2\}$	$\{v_2\}$
v'_3	C	Faulty	

First, v_3 and v'_3 cooperate: v_1 decides with the quorum $\{v_1, v_3\}$ and v'_1 with $\{v'_1, v'_3\}$, alternating every $\delta < \Delta$, each new block containing the other blockchain's latest header (Figure 3). For this example, also suppose that consensus delivers each decision to v_2, respectively v'_2, within a fixed delay of v_1, respectively v'_1. This is an additional condition on the chosen execution; *Consensus Totality* guarantees only eventual delivery. Such bounded catch-up is consistent with timely admissibility: v_2 accepts IMPORT statements through the blocking set $\{v_1\}$, without waiting for their headers to be valid at v_2 (Lemma 5.5), and similarly for v'_2. To externalize a decided B-block, however, v_2 must wait for v'_2 to externalize its C-header, which in turn requires v_2 to have externalized the previous B-block. Thus v_2 externalizes at most one B-block every 2Δ while v_1 decides one every 2δ, so v_2's lag, and the number of blocks that v_2 has decided but not externalized, grow with each block. Foreign-header validity is total in $\{v_1, v_2\}$ (Lemma 5.3), but takes ever longer to propagate. The lag is bounded when certificates transfer between validators: if v_2's C-slice were $\{v'_1\}$, for example, v_2 would externalize each block within a fixed delay of deciding it.

Now suppose that v_3 and v'_3 go silent, so that $\{v_1, v_2\}$ is the only quorum through which v_1 can decide. B keeps deciding blocks, since admissibility remains timely, but it imports no new C-header until v_2 catches up, since importing a C-header requires v_2's vote, which requires the header to be

valid at v_2. This takes time proportional to the length of the first phase and can be arbitrarily long. However, it does not violate *Liveness* or *Foreign Liveness*, which only make eventual guarantees.

The lag arises because v_2 and v_2' wait for each other's externalize messages for blocks that they have already decided. We now sketch a possible remedy. The idea is for validators to let each other know which headers they have decided but not yet externalized, and which foreign headers they are waiting for. For this, a validator that decides b broadcasts a pending message PENDING $B\ b\ \mathcal{S}$, where $\mathcal{S}$ gives its C-slices for each $C \in \text{foreign}(B)$. The message implies a condition on a set M of messages: for each foreign header y of b and of its ancestors, M contains, from every member of some slice in $\mathcal{S}$, an externalize or pending message for y or a descendant of y. A light client on a B-validator v then accepts certificates of pending messages by the following rule: a set M of messages that v has sent or received forms a certificate for a C-header c at v when M contains, from each member of one of v's C-slices, an externalize or pending message for c or a descendant of c, and M satisfies the condition of each pending message in M. Pending messages in M may thus satisfy each other's conditions, which is what resolves the mutual wait. This is safe because a well-behaved validator sends pending messages only for headers that it has decided.

In the example, the C-slice of v_2 is $\{v_2'\}$ and the B-slice of v_2' is $\{v_2\}$, so v_2's own pending messages for $b_1, \ldots, b_k$ and those of v_2' for $c_1, \ldots, c_{k-1}$ satisfy each other's conditions and form a certificate for c_{k-1} at v_2. Thus v_2 externalizes each block within a fixed delay of deciding it, in both phases, and in the second phase B imports each new C-header without waiting for v_2 to catch up. In general, members of mutually aligned intact sets externalize each block within a fixed delay of a member deciding it, provided that decisions themselves propagate within a fixed bound, as the example assumes. As with externalize messages, a certificate of pending messages implies that the well-behaved members of the slice have externalized the header or eventually will, provided that well-behaved validators rebroadcast every message they receive: a member w then eventually holds the messages that satisfied the condition of its pending message, which is just the condition for the header to be valid at w.

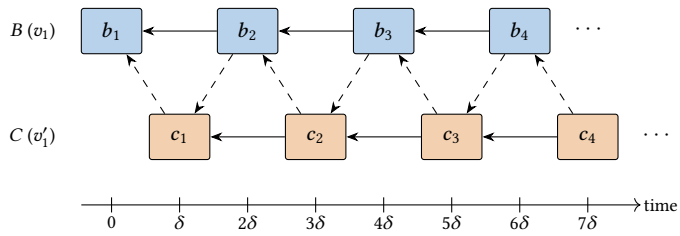


Fig. 3. Blocks decided by v_1 on B and by v_1' on C during the first phase of the mutual-bridge example, placed according to when they are decided. Solid arrows point from a block to its parent; dashed arrows point from a block to the foreign header it contains, which is the other blockchain's latest header.

6 Repairing Foreign Reorgs with Splices

In Section 5, B can never follow a foreign blockchain C again once C abandons a header that B has referenced. Suppose, as in Figure 4, that C forks at a header p, that B decides a block with C-header y, and that C then abandons y and keeps x. The C-header of every later B-block must descend from y (Sections 5.3 and 5.6.5), and none of the headers that C externalizes after the reorg, such as x' and r, does.

6.1 Splice Tables

To recover from such reorgs, B may override C's parent relation using a splice table T_B^C, a finite partial map from C-headers to C-headers used by B's light clients for C. The parent of a C-header c in B's view, written $\mathsf{parent}_B(c)$, is $T_B^C(c)$ if T_B^C maps c, and $\mathsf{parent}(c)$ otherwise. We define ancestors, descendants, and compatibility of C-headers in B's view as in Section 5.3, but following parent_B instead of parent, and we assume that parent_B is acyclic and well founded. We call the ancestors in B's view of the headers in the image of T_B^C spliced. They include headers that C abandoned and synthetic headers that C never externalized, created by B's operators along with their blocks. In Figure 4, for example, T_B^C maps r to a synthetic header y' with parent y. Then r descends from y in B's view, so B's C-header can advance from y to r, and y', y, and their ancestors in B's view are spliced.

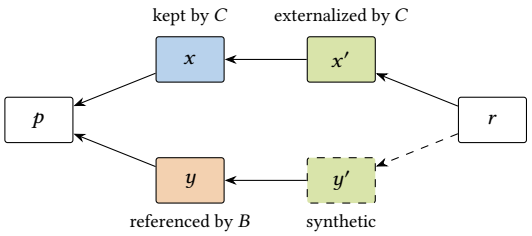


Fig. 4. Repairing a reorg of C with the splice table entry $T_B^C(r) = y'$. All headers shown are C-headers. Solid arrows point from a header to its parent; the dashed arrow points from r to its parent in B's view.

We take T_B^C as fixed and known to all B-validators. Operators distribute synthetic headers and their blocks with the table, so we assume that a B-validator that obtains a C-header also obtains its ancestors in B's view. In practice, operators add entries through coordinated protocol upgrades, ensuring compatibility with existing import votes and decided blocks, with no affected imports or block proposals in flight. We leave upgrade coordination outside our model.

We say that a C-header c is above B's splices for C when no splice applies to a strict descendant of c, either under C's parent relation or in B's view. For example, in Figure 4, r and its descendants are above B's splices, but the other headers shown are not.

6.2 Effect on the Formalization

With splices, B orders C-headers in its own view. The following use parent_B instead of parent, and thus ancestry and compatibility in B's view:

- The ordering of foreign references in application validity (Section 5.3) and the assumption on get_txset (Section 5.6.1). By the guarantees for decided headers in Section 5.6.5, the C-headers of blocks decided by a well-behaved B-validator, or by two intertwined B-validators, are thus compatible in B's view.
- $\mathsf{is_parent}(c_P, c)$, which holds when $c_P = \mathsf{parent}_B(c)$, so that contracts check ancestry in B's view (Section 5.6.2).
- The statements IMPORT C c_P c, in which $c_P = \mathsf{parent}_B(c)$, and thus the chain of imported C-headers and the block manager's rules (Section 5.6).
- *Foreign Liveness*, and the proofs of Lemma 5.5 and Theorem 5.7.

Elsewhere, ancestry and compatibility of C-headers still follow C's parent relation. In particular, since externalizing a C-header externalizes its ancestors on C, a certificate for r in Figure 4 covers x', x, and p, but neither y' nor y.

Since spliced headers need not have certificates, validity gains a case: a C-header c is valid at a validator v when $c = g_C$, c is spliced, or v's light client has obtained a certificate for c at v. The set of headers valid at v then remains closed under ancestry in B's view, as Section 5.6.2 requires: from a header with a certificate at v, parent_B follows headers that the certificate covers until it first follows an entry of T_B^C, whose target and its ancestors in B's view are spliced.

Like g_C, spliced headers are thus valid at every validator. Unlike g_C, they are not imported initially: B imports them by federated voting, like other C-headers, so that the C-headers imported at a validator still form a chain.

6.3 Guarantees with Splices

For a fixed splice table, Theorem B.1, Lemma 5.3, Lemma 5.5, Lemma 5.6, Theorem 5.7, and their proofs carry over, with spliced headers handled like genesis headers: both are valid at every validator by definition. Theorem B.2 and Theorem 5.4 change, however, since C need not have externalized a spliced header. If a set V' of B-validators is C-aligned with a set I_C of C-validators, then, as in the proof of Lemma 5.3, every C-header valid at a member of V' is now spliced or was externalized by a member of I_C (Theorem B.3 in Section B).

In B's view, Theorem 5.4 needs a new hypothesis. In Figure 4, B may import the spliced header y, after which no C-header of a later B-block descends from x' in B's view, even if a member of I_C externalized x'. We therefore start from a C-header above B's splices, so that the two parent relations agree on its strict descendants:

Corollary 6.1. *Under the hypotheses of Lemma 5.3, suppose moreover that a member of I_B externalizes a B-block containing a C-header c_0 above B's splices for C. If a member of I_C externalizes a descendant c of c_0, then every member of I_B eventually externalizes a B-block with a C-header that is a descendant of c in B's view. The same holds with B and C exchanged.*

Proof. Since c_0 is above B's splices, its descendants are the same in both views, and none is spliced: every spliced header has a strict descendant in B's view at which a splice applies. By Lemma 5.3 and alignment, foreign-header validity is total in I_B, and c eventually becomes valid at every member of I_B. Every descendant d of c_0 valid at a member of I_B is not spliced, so, as above, it was externalized by a member of I_C. By *Consistency* of C, d is compatible with c, in both views. Applying *Foreign Liveness* to the assumed B-block gives the conclusion. □

Like Theorem 5.4, it generalizes to any number of blockchains.

Stating *Foreign Liveness* relative to an externalized block lets B reach such a c_0. In Figure 4, with $T_B^C(r) = y'$ as the only entry and under the hypotheses of Lemma 5.3, suppose that a member of I_B externalizes a B-block b with C-header y, and that a member of I_C externalizes r, so that r becomes valid at the members of I_B. Every non-spliced C-header valid at a member of I_B was externalized by a member of I_C (see above), and is thus compatible with r. So the valid descendants of y in B's view are y, y', and descendants of r; in particular, x and x' remain valid but do not descend from y in B's view. *Foreign Liveness*, applied to b and to $c = r$, then gives every member of I_B a B-block whose C-header descends from r, which can serve as c_0.

6.4 Discussion

A splice does not by itself repair the damage of a reorg. If B's application has acted upon events of y, such as deposits into a bridge contract, that C no longer contains, the communities of B and C must agree, informally, on a remediation plan. The splice and any synthetic headers implement B's side of the plan while preserving B's decided history. In Figure 4, for example, C can externalize x', whose block replays the events of y that x lacks, followed by r, and B's operators can create y',

whose block replays the events of x that y lacks. Then the ancestors of r in B's view, $r, y', y, p, \ldots$, carry the same events as its ancestors, $r, x', x, p, \ldots$, each exactly once. If some events of y cannot be replayed on C, for example because they include a double spend, the communities can instead agree on another plan, such as burning the excess tokens on B on a schedule.

Each blockchain that bridges from C decides independently whether to splice: one that referenced x needs no splice to follow C to r.

7 Conclusion

The notion of quorum slice from FBA consensus can be extended across different replicated state machines, allowing multiple blockchains to track each other's block headers. We present a technique that embeds light clients for foreign blockchains at every local validator, where each light client is configured with quorum slices of validators of the foreign blockchain. A light client considers a remote block header valid when the header has been unanimously externalized by one of the light client's quorum slices. Before referencing the tip of a foreign blockchain in a block proposed to the local consensus protocol, local validators must vote to import the tip. Before externalizing a local block, validators must ensure that any foreign block headers referenced in the local block are valid.

Our technique exposes foreign block headers through an API allowing smart contracts to verify foreign block headers and their parents. This API is precisely what is required to implement bridges that move assets across blockchains. Unlike prior approaches for proof-of-work and proof-of-stake blockchains, FBA bridging enjoys both asymptotic security (in the security parameters of digital signatures) and liveness under partial synchrony.

References

[1] Vitalik Buterin. 2013. Bitcoin Network Shaken by Blockchain Fork. *Bitcoin Magazine* (March 13 2013). https://bitcoinmagazine.com/technical/bitcoin-network-shaken-by-blockchain-fork-1363144448.

[2] Vishal Chawla. 2022. Harmony's $100 million hacker took control of its multi-signature wallet, analysts say. *The Block* (June 24 2022). https://www.theblock.co/news/regulation/2022-06-24-harmonys-100-million-hacker-took-control-of-its-multi-signature-wallet-analysts-say-154029.

[3] Tim Copeland. 2023. Multichain ceases operations as CEO's sister is detained in China while holding $220 million of funds. *The Block* (July 14 2023). https://www.theblock.co/news/ecosystems/2023-07-14-multichain-ceases-operations-as-ceos-sister-is-detained-in-china-while-holding-220-million-of-funds-239589.

[4] Tom Dickman. retrieved 2026. PoW 51% Attack Cost. https://www.crypto51.app/.

[5] Stellar Development Foundation. Retrieved October 2026. Tier 1 Organizations. https://developers.stellar.org/docs/validators/tier-1-orgs.

[6] Christopher Goes. 2020. The Interblockchain Communication Protocol: An Overview. arXiv:2006.15918 [cs.DC] https://arxiv.org/abs/2006.15918

[7] Aggelos Kiayias, Andrew Miller, and Dionysis Zindros. 2020. Non-interactive Proofs of Proof-of-Work. In *Financial Cryptography and Data Security*, Joseph Bonneau and Nadia Heninger (Eds.). Springer International Publishing, 505–522.

[8] Marta Lokhava, Giuliano Losa, David Mazières, Graydon Hoare, Nicolas Barry, Eli Gafni, Rafał Malinowsky, and Jed McCaleb. 2019. Fast and Secure Global Payments with Stellar. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. Huntsville, Ontario.

[9] David Mazières. 2015. The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus. (April 2015). https://www.stellar.org/papers/stellar-consensus-protocol.pdf

A Omitted proofs

Theorem 5.1 (Cascade). *Let I be an intact set, Q a quorum that intersects I, and S a set that includes Q. If $I \not\subseteq S$, then some member of $I \setminus S$ is blocked by $I \cap S$.*

Proof. Suppose that $I \not\subseteq S$ and that no member of $I \setminus S$ is blocked by $I \cap S$. Then each member of $I \setminus S$ has a slice disjoint from $I \cap S$, whose intersection with I is included in $I \setminus S$, so $I \setminus S$ is a non-empty quorum in $\Sigma \restriction I$. Moreover, each member of $Q \cap I$ has a slice included in Q, so $Q \cap I$ is a

non-empty quorum in $\Sigma \upharpoonright I$. Since I has quorum intersection despite its complement, these two quorums intersect, which contradicts $Q \subseteq S$. □

Lemma 5.2 (Alignment). *A set V' of B-validators is C-aligned with a set I_C of C-validators if and only if V' is well-behaved, I_C is C-intact, and every member v of V' satisfies:*

(1) *Every C-slice of v intersects I_C.*
(2) *Some C-slice of v is included in I_C.*

Proof. First suppose that $J = V' \cup I_C$ is intact in Σ_B^C. Its members are well-behaved. Since all slices in Σ_B^C are non-empty and contain only C-validators, the non-empty quorum J must contain a member of I_C. Each member of I_C has a slice included in J, hence in I_C, so I_C is a quorum in Σ_C. Quorums of its members in $\Sigma_C \upharpoonright I_C$ are also quorums of those members in $\Sigma_B^C \upharpoonright J$, and therefore intersect. Thus I_C is C-intact.

Each $v \in V'$ has a C-slice included in J, hence in I_C. If some C-slice of v were disjoint from I_C, it would be empty after projection to J. Then $\{v\}$ and I_C would be disjoint quorums of members of J in $\Sigma_B^C \upharpoonright J$, contradicting intactness.

Conversely, suppose that the stated conditions hold. Then $J = V' \cup I_C$ is non-empty, well-behaved, and a quorum, since every member has a slice included in I_C. In $\Sigma_B^C \upharpoonright J$, all slices lie in I_C and are non-empty: for members of V' this follows from the first slice condition, and for members of I_C from its intactness, since a member of I_C with a slice disjoint from I_C would have $\emptyset$ as a quorum in $\Sigma_C \upharpoonright I_C$. Hence every quorum Q of a member of J contains a member of I_C, and $Q \cap I_C$ is a non-empty quorum in $\Sigma_C \upharpoonright I_C$. Since I_C is intact, any two such intersections, and thus the original quorums, intersect. So J is intact in Σ_B^C. □

Lemma 5.3. *Suppose that B and C are each other's only foreign blockchains, and that a B-intact set I_B and a C-intact set I_C are mutually aligned. Then every B-header externalized by a member of I_B is eventually externalized by every member of I_B, and foreign-header validity is total in I_B. The same holds for C and I_C.*

Proof. Let X be B or C, and Y the other. By Lemma 5.2, every member of I_X has a Y-slice included in I_Y, and every Y-slice of a member of I_X intersects I_Y. Thus, if a Y-header $y \neq g_Y$ is valid at a member w of I_X, its certificate contains an externalize message externalizing y from a member of I_Y. That member is well-behaved and therefore externalized y.

We now show, simultaneously for $X = B$ and $X = C$, by induction on the causal order of externalization events, that if a member w of I_X externalizes an X-header x, then every member of I_X eventually externalizes x. This order is well founded, since only finitely many events occur in any bounded period of time (Section 5.1). If $x = g_X$, every member of I_X externalizes x from the start. Otherwise, by *Validity*, x is application-valid, and each foreign header y of x and of its ancestors, which is a Y-header, is valid at w when w externalizes x. If $y = g_Y$, then y is valid at every member of I_X by definition. Otherwise, by the above, a member of I_Y externalized y causally before w externalized x. By the induction hypothesis, every member of I_Y eventually externalizes y, and thus broadcasts an externalize message externalizing y. Since every member of I_X has a Y-slice included in I_Y, y eventually becomes valid at every member of I_X. Since x has finitely many ancestors, x thus eventually becomes valid at every member of I_X, and, by *Externalization Totality*, every member of I_X eventually externalizes x.

Finally, foreign-header validity is total in I_X: g_Y is valid at every member of I_X, and if another Y-header y is valid at a member of I_X, then, by the above, a member of I_Y externalized y, so every member of I_Y eventually does, and y eventually becomes valid at every member of I_X. □

Theorem 5.4 (Liveness composition). *Under the hypotheses of Lemma 5.3:*

(1) *The guarantees of Liveness and Foreign Liveness apply to I_B and to I_C.*
(2) *If a member of I_C externalizes a C-header c, then every member of I_B eventually externalizes a B-header whose C-header is a descendant of c. The same holds with B and C exchanged.*

Proof. By Lemma 5.3, foreign-header validity is total in I_B and in I_C, which gives the first part.

For the second part, let c be a C-header externalized by a member of I_C. As shown in the proof of Lemma 5.3, every member of I_B has a C-slice included in I_C, and every C-header $d \neq g_C$ valid at a member of I_B was externalized by a member of I_C; so was g_C, which every member of I_C externalizes from the start. Since I_C is C-intact, hence intertwined, *Consistency* of C implies that every C-header valid at a member of I_B is compatible with c. In particular, c is compatible with g_C, which has no parent, so c descends from g_C. Moreover, by Lemma 5.3, every member of I_C eventually externalizes c, and thus broadcasts an externalize message externalizing c, so c eventually becomes valid at every member of I_B. We can thus apply *Foreign Liveness* to I_B, in which foreign-header validity is total, with $b = g_B$, which every member of I_B externalizes from the start and which has no C-header, so that $c_b = g_C$. So every member of I_B eventually externalizes a B-header whose C-header is a descendant of c. The same argument applies with B and C exchanged. □

Lemma 5.5. *Let I be a B-intact set and c a C-header imported at a member of I at time t. Then c is valid at a member of I, and c is imported at every member of I by time $t + |I|\Delta$.*

Proof. The claim holds for $c = g_C$ by initialization. Suppose that $c \neq g_C$.

Every blocking set of a member of I intersects I, since I is a quorum. So the first member w of I to accept an IMPORT statement does not accept it because a blocking set has. Instead, w belongs to a quorum Q whose members have voted for or claimed to accept the statement. No member of $Q \cap I$ has yet accepted it, so every member of $Q \cap I$ voted for it, including w. For IMPORT C parent(c) c, this means that c is valid at w, which proves the first part. It also follows that members of I never accept two conflicting statements: the quorums through which the first members of I to accept them did so would each intersect I in a non-empty quorum of $\Sigma_B \restriction I$, all of whose members voted for the corresponding statement. Since I is intact, these quorums intersect in a member of I, which would have voted for both.

For the second part, consider an IMPORT statement that a member u of I confirmed by time t through a quorum Q that contains u. The members of $Q \cap I$ accepted the statement by time t. Let A_k be the set of members of I that have accepted the statement by time $t + k\Delta$, so that $Q \cap I \subseteq A_k$. Each member of A_k has broadcast its acceptance by then, so, if $A_k \neq I$, Theorem 5.1 applied to $S = Q \cup A_k$ gives a member of $I \setminus A_k$ that is blocked by A_k and thus accepts the statement by time $t + (k+1)\Delta$, whether or not c is valid at it; by the above, it has not accepted a conflicting statement. Since $A_0 \supseteq Q \cap I$ is non-empty, $A_{|I|-1} = I$, and, since I is a quorum, every member of I confirms the statement by time $t + |I|\Delta$. This applies to the IMPORT statements along the parent path from c back to g_C, excluding g_C. The member of I that imported c confirmed all these statements by time t, so their confirmations propagate in parallel within the same bound. Since g_C is imported initially, every member of I imports c by time $t + |I|\Delta$. □

Lemma 5.6. *Admissibility is timely in every B-intact set I, with $D = |I|\Delta + \Delta_{val}$.*

Proof. Suppose that a B-header b is admissible at a member u of I at time t. Then b is application-valid, which does not depend on the validator, and each foreign header of b and of its ancestors is imported at u by time t, hence, by Lemma 5.5, at every member v of I by time $t + |I|\Delta$. By time t, u has also broadcast b and the data needed to check its application validity. Every member v receives them by $t + \Delta$, which is no later than $t + |I|\Delta$ since I is non-empty. It completes the application-validity check within Δ_{val} (Section 5.6.3), so b is admissible at every member by $t + |I|\Delta + \Delta_{val}$. □

Theorem 5.7. *If the application satisfies the assumptions of Section 5.6.1, the consensus layer provides the guarantees of Section 5.6.3, and well-behaved B-validators follow the rules of Section 5.6, then B satisfies the properties of Section 5.4.*

Proof. A well-behaved validator v decides only headers that its consensus layer decides and that are admissible at v, hence application-valid, and it externalizes only headers that are already valid at v. This gives *Validity*. Since a validator externalizes only headers that it has decided, *Consistency* follows from *Consensus Consistency*.

For the other properties, consider a B-intact set I. By Lemma 5.6, admissibility is timely in I. We first show that if the consensus layer of a member of I decides a header b, then every member v of I eventually decides b. By *Consensus Validity*, b is admissible at a member of I, hence eventually at v, and, by *Consensus Totality*, the consensus layer of v eventually decides b. Then v externalizes b once b is valid at v, that is, once the foreign headers of b and of its ancestors are valid at v. *Externalization Totality* follows, since a member of I externalizes only headers that its consensus layer decided.

For *Liveness*, suppose that foreign-header validity is total in I, let R be a finite set of transactions that every member of I eventually receives, and consider a member v of I and a foreign header c of a header that v decides, or of one of its ancestors. Since c is imported at v, Lemma 5.5 implies that c is valid at a member of I. Since foreign-header validity is total in I, c eventually becomes valid at v. So v eventually externalizes every header that it decides. Initially, g_B is admissible at every member of I, and each light client has imported its foreign genesis header. The block manager can extend any admissible parent using these or later imported headers, since get_txset eventually returns a valid transaction set. By *Consensus Liveness*, the consensus layers of members of I decide infinitely many headers, each proposed by a member of I and thus with some parent b_P, foreign headers H, and a transaction set returned by $\text{get_txset}(b_P, H)$ at that member. Choose a time t_R by which every member of I has received all of R. Only finitely many calls to get_txset began at or before t_R, since only finitely many events occur in a bounded period, and each call constructs one proposal. Thus only finitely many of the decided headers can come from those calls, even if the calls return after t_R. Every remaining header b comes from a call made after the caller received all of R, so $\pi_B(b, R)$ holds by the assumption on get_txset and antitonicity. There are infinitely many such headers, and, by the above, every member of I decides and externalizes each of them.

For *Foreign Liveness*, take b, c_b, and c as in the property. By the above, every member of I eventually decides b. Thus every member eventually imports c_b: it is the C-header of b, or g_C if b has none, in which case it is imported initially. By the hypothesis of *Foreign Liveness*, c descends from c_b. If $c = c_b$, every member's latest imported C-header already descends from c. Otherwise, consider a strict descendant d of c_b that is an ancestor of c. Since c becomes valid at a member of I and foreign-header validity is total in I, c eventually becomes valid at every member, and so does d (Section 5.6.2). The C-header of a statement that conflicts with $\textsc{import}\ C\ \text{parent}(d)\ d$ descends from c_b and is incompatible with c, so it is never valid at a member of I, and no member of I votes for that statement or, by the proof of Lemma 5.5, accepts it. So every member of I eventually votes for $\textsc{import}\ C\ \text{parent}(d)\ d$, and, since I is a quorum, eventually confirms it. Applying this argument along the finite parent path from c_b to c, every member of I eventually imports c. In either case, choose a time t_c by which every member's latest imported C-header descends from c. Every call to get_txset begun after t_c therefore uses a C-header descending from c. Only finitely many calls began at or before t_c, so only finitely many proposed headers can come from those earlier calls. By *Consensus Liveness*, the consensus layers of members of I decide infinitely many headers proposed by members of I, hence one constructed by a call begun after t_c. Its C-header descends from c, and, as in the proof of *Liveness*, every member of I externalizes it. □

B Additional properties

Theorem B.1. *Suppose that the members of a B-intact set I use the same C-slices for each foreign blockchain C and relay the externalize messages that their light clients receive. Then foreign-header validity is total in I. Hence, regardless of the behavior of foreign validators, for every finite set R of transactions that every member of I eventually receives, every member of I externalizes infinitely many headers b such that $\pi_B(b, R)$, and the guarantee of Foreign Liveness applies to I.*

Proof. The genesis header g_C is valid at every member by definition. If another C-header c is valid at a member v of I, then v's light client has received externalize messages for c or a descendant of c from each member of one of its C-slices. Since v relays these messages, every member of I eventually receives them and, since it has the same C-slices, eventually considers c valid. The rest follows from *Liveness* and *Foreign Liveness*. □

Theorem B.2. *Let V' be a set of B-validators that is C-aligned with a set I_C of C-validators. If a member of V' externalizes a B-header whose C-header is c, then a member of I_C externalized c, and thus c is compatible with every C-header externalized by a member of I_C.*

Proof. Let v be the member of V'. By Lemma 5.2, v is well-behaved, so, by *Validity*, c is valid at v. If $c = g_C$, every member of I_C externalizes c from the start. Otherwise, v's light client has received externalize messages for c or a descendant of c from each member of some C-slice S of v. By Lemma 5.2, S contains a member of I_C, which is well-behaved and thus externalized c. Finally, since I_C is C-intact, hence intertwined, *Consistency* of C implies that c is compatible with every C-header externalized by a member of I_C. □

With splices (Section 6), Theorem B.2 becomes:

Theorem B.3. *Let V' be a set of B-validators that is C-aligned with a set I_C of C-validators. If a member of V' externalizes a B-header whose C-header is c, then c is spliced or a member of I_C externalized c, in which case c is compatible with every C-header externalized by a member of I_C.*

Proof. As for Theorem B.2, except that c may also be valid at v because it is spliced, in which case nothing follows about I_C. □

If the members of V' are moreover intertwined in B, the C-headers of the B-blocks that they externalize are also compatible with each other in B's view (Section 6.2).